Programmazione parallela su architetture GPU

Appunti personali

Università degli Studi di Catania
Dipartimento di Matematica e Informatica
Corso di Laurea Triennale in Informatica (L-31)

Autori
Emanuele Galiano
Sofia Lo Vecchio

Anno accademico
Anno Accademico 2025/2026

Versione: 17 marzo 2026

Prefazione

Queste dispense sono nate come appunti personali per il corso di *Programmazione parallela su architetture GPU– Appunti personali* tenuto presso il Dipartimento di Matematica e Informatica dell'Università di Catania tenuto dal Prof. Giuseppe Bilotta nell'anno accademico Anno Accademico 2025/2026. L'obiettivo principale di questo materiale è stato fornire una risorsa personale di studio sulla parte teorica del corso, raggruppando concetti chiave, definizioni, teoremi, immagini ed esempi in unico documento.

Questo file non deve essere visto come un testo ufficiale o completo sull'argomento, in quanto potrebbero esserci errori, omissioni o imprecisioni. Invito pertanto chi legge questo documento a consultare le dispense ufficiali del corso proposte dal docente ed eventuali testi di riferimento consigliati. Inoltre, invito chiunque noti errori o abbia suggerimenti a contattarmi via email:

- **Email personale**: galianoo.emanuele@gmail.com,
- **Email universitaria**: emanuele.galiano@studium.unict.it;

oppure se ancora presente online, aprire una **issue** o una **pull request** nel repository GitHub associato a queste dispense:

`https://github.com/emanuelegaliano/GPU-Parallel-Programming`

In particolare, all'interno del repository GitHub sono presenti il codice open-source dei sorgenti LaTeX utilizzati per generare queste dispense, oltre a tutti i file di supporto come immagini e codici per la generazione di alcuni grafici presenti nel testo.

Licenza

Questo documento, intitolato *Programmazione parallela su architetture GPU – Appunti personali*, è distribuito sotto licenza **Creative Commons Attribution–ShareAlike 4.0 International (CC BY-SA 4.0)**.

La presente licenza è stata scelta con l'obiettivo di favorire la diffusione, la condivisione e il riutilizzo del materiale didattico, garantendo al contempo il riconoscimento degli autori originali e la preservazione della natura open delle opere derivate.

Autori: Emanuele Galiano
Sofia Lo Vecchio
Affiliazione: Università degli Studi di Catania – Dipartimento di Matematica e Informatica
Anno accademico: Anno Accademico 2025/2026

Diritti concessi

In conformità con i termini della licenza Creative Commons BY-SA 4.0, è consentito a chiunque di:

- copiare e ridistribuire il materiale in qualsiasi mezzo o formato;
- adattare, modificare e trasformare il materiale;
- utilizzare il materiale anche per scopi commerciali.

Tali diritti sono concessi a titolo gratuito e non possono essere revocati, purché siano rispettate le condizioni indicate nella sezione seguente.

Condizioni

L'utilizzo del materiale è subordinato al rispetto delle seguenti condizioni:

- **Attribuzione (BY):** deve essere fornita un'adeguata attribuzione dell'opera, citando **titolo**, **autori** e, ove possibile, la **fonte**. L'attribuzione deve essere effettuata in modo ragionevole e non tale da suggerire che gli autori originali approvino l'uso o le modifiche apportate.
- **Condividi allo stesso modo (SA):** nel caso in cui il materiale venga modificato, trasformato o utilizzato per creare opere derivate, tali opere devono essere distribuite sotto la *stessa licenza* Creative Commons Attribution–ShareAlike 4.0 International.

Non è consentito applicare termini legali o misure tecnologiche che limitino giuridicamente altri utenti dall'esercitare i diritti concessi dalla licenza.

Assenza di garanzia

Il materiale è fornito *"così com'è"*, senza garanzie di alcun tipo, esplicite o implicite. In particolare, gli autori non garantiscono l'accuratezza, la completezza o l'assenza di errori nel contenuto del documento e declinano ogni responsabilità per eventuali danni derivanti dall'uso del materiale.

Testo completo della licenza

Il testo legale completo della licenza Creative Commons Attribution–ShareAlike 4.0 International è disponibile al seguente indirizzo:

`https://creativecommons.org/licenses/by-sa/4.0/`

Copyright © 2026
Emanuele Galiano
Sofia Lo Vecchio

Indice

Licenza **v**
Diritti concessi . . . v
Condizioni . . . v
Assenza di garanzia . . . vi
Testo completo della licenza . . . vi

1 Introduzione **1**
1.1 Calcolo parallelo . . . 1
1.2 Limiti del calcolo parallelo . . . 1
1.2.1 Caso ideale di parallelizzazione . . . 1
1.2.2 Legge di Amdahl . . . 2
1.2.3 Legge di Gustafson . . . 3
1.3 Tipi di parallelismo . . . 4
1.3.1 Task-based parallelism . . . 4
1.3.2 Data-based parallelism . . . 5
1.4 Hardware per il calcolo parallelo . . . 6
1.4.1 Shared memory . . . 7
1.4.2 Distributed memory . . . 7
1.4.3 Ibrida . . . 7
1.5 Concorrenza, latenza e banda . . . 8
1.6 Richiesta e offerta di calcolo parallelo . . . 9
1.6.1 Domanda di calcolo parallelo . . . 9
1.6.2 Offerta di calcolo parallelo . . . 9
1.6.3 Videogiochi e GPU . . . 10
1.7 Storia delle GPU . . . 11
1.7.1 Dai primi adattatori video alle prime schede programmabili . . . 11
1.7.2 La transizione verso la grafica 3D . . . 12
1.7.3 Dalla GPU al GPGPU . . . 12
1.8 GPU moderne . . . 12
1.8.1 GPU e GPU: performance e costi . . . 12
1.9 Parallelizzazione . . . 13
1.9.1 Concetto generale di parallelizzazione . . . 13
1.9.2 Algoritmi con parallelismo parziale . . . 14
1.9.3 Parallelismo in hardware . . . 14
1.9.4 Parallelismo in software . . . 14

2 Fondamenti di GPGPU **17**
2.1 Architettura di un programma GPGPU 17
2.1.1 Kernel, work-item e work-group 17
2.2 Stream-Processing 18
2.2.1 Kernel e dati 18
2.2.2 Esempio: Mandelbrot fractal 18
2.3 Memoria 19
2.4 Struttura di un programma GPGPU 20
2.4.1 Pipeline di esecuzione 20
2.4.2 Comunicazione host-device 21
2.4.3 Sorgente singolo e separato 21
2.5 Stream Computing 23

Capitolo 1

Introduzione

1.1 Calcolo parallelo

Il calcolo parallelo è un paradigma di programmazione che consente di eseguire più operazioni contemporaneamente, sfruttando la potenza di elaborazione di più unità di calcolo. A differenza dello pseudoparallelismo, che simula l'esecuzione simultanea di più processi su un singolo processore, il calcolo parallelo utilizza effettivamente più processori o core per eseguire compiti in parallelo.

Questo paradigma è utile perché riduce i tempi di esecuzione e aumenta la dimensione dei dati processabili, consentendo di affrontare problemi più complessi e di ottenere risultati più rapidamente.

1.2 Limiti del calcolo parallelo

1.2.1 Caso ideale di parallelizzazione

Per comprendere i limiti del calcolo parallelo è utile partire da un modello ideale. Supponiamo che un programma sia completamente parallelizzabile, cioè che tutto il lavoro possa essere suddiviso tra più unità di calcolo senza alcuna parte sequenziale.

Se T_0 indica il tempo di esecuzione del programma in modalità sequenziale e N il numero di unità di calcolo disponibili, il tempo di esecuzione ideale del programma parallelo sarebbe:

$$T_L(N) = \frac{T_0}{N}$$

In questo caso il tempo di esecuzione diminuisce linearmente all'aumentare del numero di processori. Lo speed-up ottenuto è quindi:

$$S_L(N) = \frac{T_0}{T_L(N)} = N$$

Questo significa che, in condizioni ideali, raddoppiare il numero di unità di calcolo dimezza il tempo di esecuzione. In tale modello non esiste quindi un limite teorico allo speed-up ottenibile aumentando il numero di processori.

1.2.2 Legge di Amdahl

Il calcolo parallelo non può essere applicato a tutti i problemi, e anche quando è possibile, esiste un limite alla velocità di esecuzione che può essere raggiunta. Questo limite è descritto dalla legge di Amdahl:

> *La velocità massima di un programma parallelo è limitata dalla frazione di codice che deve essere eseguita in modo sequenziale.*

Adamahl ha dimostrato che se una frazione p del programma può essere parallelizzata, allora la velocità massima teorica che si può ottenere è data da:

$$T_A(N) = \frac{pT_0}{N} + (1-p)T_0$$

dove T_0 è il tempo di esecuzione del programma sequenziale e N è il numero di processori. Si può anche esprimere il massimo speed-up come:

$$S_A(N) = \frac{T_0}{T_A(N)} = \frac{1}{p/N + (1-p)} = \frac{N}{p + N(1-p)}$$

Da questa formula si evince che, anche con un numero infinito di processori, il massimo speed-up è

$$\lim_{N\to\infty} S_A(N) = \frac{1}{1-p}$$

Il che significa che se una parte significativa del programma è sequenziale, il miglioramento ottenuto con il calcolo parallelo sarà limitato, indipendentemente dal numero di processori utilizzati.

p	2	4	100	200	400	∞
10%	1.05	1.08	1.11	1.11	1.11	1.11
25%	1.14	1.23	1.33	1.33	1.33	1.33
50%	1.33	1.60	1.98	1.99	1.99	2.00
75%	1.60	2.29	3.88	3.94	3.97	4.00
99%	1.98	3.88	50.2	66.9	80.2	100

Tabella 1.1: Speed-up teorico secondo la legge di Amdahl al variare della frazione parallelizzabile p e del numero di processori. Come si può notare all'aumentare delle unità di calcolo, lo speed-up tende a un limite massimo che dipende dalla frazione di codice sequenziale.

Ottimizzazione della legge di Amdahl. Ipotizzando che il lavoro totale possa essere diviso in parti multiple $p_1, p_2, \ldots, p_r$ e ognuna abbia uno speed-up $s_1, s_2, \ldots, s_r$, allora lo speed-up totale è dato da:

$$S = \frac{1}{\sum_{i=1}^{r} \frac{p_i}{s_i}}$$

In questo caso ci si potrebbe chiedere se convenga concentrarsi sull'ottimizzazione della parte con lo speed-up maggiore s_i oppure su quella che rappresenta la porzione più grande del programma p_i.

Consideriamo ad esempio un programma suddiviso in due parti, con $p_1 = 0.25$ e $p_2 = 0.75$. Supponiamo inoltre che la prima parte possa essere accelerata con uno speed-up $s_1 = 20$, mentre la seconda con uno speed-up $s_2 = 5$.

Se si ottimizza solo la prima parte si ottiene:

$$S = \frac{1}{\frac{0.25}{20} + 0.75} \approx 1.31$$

Se invece si ottimizza solo la seconda parte:

$$S = \frac{1}{0.25 + \frac{0.75}{5}} = 2.5$$

Infine, ottimizzando entrambe le parti:

$$S = \frac{1}{\frac{0.25}{20} + \frac{0.75}{5}} \approx 6.15$$

Questo esempio mostra che, per ottenere il miglior miglioramento complessivo, conviene concentrarsi sull'ottimizzazione della parte che occupa la porzione maggiore del tempo di esecuzione del programma.

1.2.3 Legge di Gustafson

La legge di Amdahl si concetra su quanto è possibile massimizzare lo speed-up assumendo che il lavoro sia fisso (strong scaling).

La legge di Gustafson, invece, si concentra su quanto è possibile aumentare la dimensione del lavoro mantenendo costante il tempo di esecuzione (weak scaling).

> *Se una frazione p del lavoro può avere uno speed-up di fattore s, allora il lavoro totale che può essere eseguito in un tempo costante è dato da:*
>
> $$S_G = \frac{W_S}{W_0} = 1 - p + sp$$
>
> dove W_S è il lavoro totale eseguito in parallelo, W_0 è il lavoro eseguito in modalità sequenziale.

Esempio. Supponiamo che l'80% del lavoro possa essere parallelizzato con uno speed-up di 20, allora nello stesso tempo di esecuzione del programma sequenziale, è possibile eseguire un lavoro totale di:

$$S_G = 1 - 0.8 + 20 \cdot 0.8 = 16.2$$

Non linearità dello speed-up. La legge di Gustafson mostra che, aumentando la dimensione del lavoro, è possibile ottenere uno speed-up maggiore, anche se la frazione di codice parallelizzabile rimane costante. Tuttavia, è importante notare che lo speed-up non aumenta linearmente con la dimensione del lavoro, ma dipende dalla frazione di codice parallelizzabile e dallo speed-up ottenuto su quella frazione.

Esempio. Supponiamo che W sia $O(D^3)$ (dove D è la dimensione dei dati). Allora $16\times$ più lavoro corrisponde a un aumento della dimensione dei dati di $\sqrt[3]{16} \approx 2.52$ volte.

1.3 Tipi di parallelismo

Un problema può essere parallelizzato in diversi modi, a seconda della natura del lavoro da svolgere e delle dipendenze tra le operazioni. I principali tipi di parallelismo sono:

- Data-based: le operazioni possono essere eseguite in parallelo su diversi dati, ad esempio l'elaborazione di un array o di una matrice.
- Task-based: le operazioni possono essere eseguite in parallelo su diversi compiti, ad esempio l'esecuzione di più funzioni o processi indipendenti.

1.3.1 Task-based parallelism

Questo tipo di parallelismo si basa sull'esecuzione di compiti indipendenti in parallelo. Ad esempio, se un programma deve eseguire più funzioni che non dipendono l'una dall'altra, queste possono essere eseguite contemporaneamente su processori diversi. Si preferisce questo tipo di parallelismo se:

- Il problema può essere suddiviso in compiti (meglio indipendenti e di dimensione simile).
- Numero di task $\approx$ numero di processori disponibili.
- Le unità che possiedono diversi task possono comunicare in modo semplice.

Esempio: pipeline di elaborazione. Consideriamo un carico di lavoro composto da tre tipi di task applicati a più dataset (A, B, C, ...): la **lettura** dei dati (R), l'**elaborazione** o calcolo (C) e la **scrittura** dei risultati (W). Per semplicità assumiamo tempi di esecuzione fittizi costanti pari a $t_R = 2$ per la lettura, $t_C = 3$ per l'elaborazione e $t_W = 2$ per la scrittura. Nel caso **dipendente** ogni dataset deve rispettare la sequenza $R \to C \to W$, cioè i dati devono essere letti prima di poter essere elaborati e i risultati possono essere scritti solo dopo il completamento del calcolo. Nel caso **indipendente**, invece, i task possono essere schedulati senza vincoli di dipendenza e quindi distribuiti più liberamente tra le unità di calcolo.

Nel caso più semplice, mostrato in Figura 1.1, l'esecuzione avviene su una sola unità di calcolo. Tutti i task vengono quindi eseguiti in modo completamente seriale e ogni dataset attraversa l'intera sequenza di operazioni prima che il successivo possa iniziare. L'ordine di esecuzione diventa quindi $R_A, C_A, W_A, R_B, C_B, W_B, \ldots$, rappresentando il caso base senza alcuna forma di parallelismo.

Introducendo una seconda unità di calcolo e mantenendo le dipendenze tra task (Figura 1.2), è possibile ottenere una parziale sovrapposizione delle operazioni. Ad esempio, mentre una unità legge il dataset successivo, l'altra può elaborare quello precedente. Tuttavia la presenza delle dipendenze impone comunque dei vincoli sull'ordine di esecuzione e genera inevitabilmente alcuni periodi di inattività.

Se i task sono indipendenti (Figura 1.3), lo scheduler può distribuire le operazioni tra le due unità con maggiore libertà. Questo permette di riempire meglio i tempi morti e di ottenere

un utilizzo più efficiente delle risorse di calcolo, aumentando quindi il grado di parallelismo effettivamente sfruttabile.

Aumentando il numero di dataset, come mostrato in Figura 1.4, la pipeline tende a stabilizzarsi anche nel caso dipendente. Le unità restano occupate più a lungo e il throughput complessivo migliora, ma il cammino critico imposto dalle dipendenze continua comunque a limitare lo speed-up massimo ottenibile.

Utilizzando tre unità di calcolo con task dipendenti (Figura 1.5) l'aggiunta di risorse non produce necessariamente un'accelerazione proporzionale. Il vincolo $R \rightarrow C \rightarrow W$ per ogni dataset rende infatti difficile mantenere tutte le unità occupate contemporaneamente, causando possibili squilibri nel carico di lavoro.

Nel caso indipendente con tre unità (Figura 1.6), invece, il parallelismo può essere sfruttato in modo più efficace. I task possono essere distribuiti dinamicamente tra le unità e il carico di lavoro risulta più bilanciato, permettendo di avvicinarsi ad un utilizzo più efficiente delle risorse disponibili.

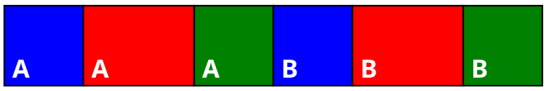

Figura 1.1: Task-based: una sola unità (esecuzione seriale).

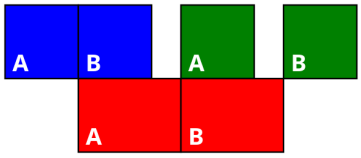

Figura 1.2: Task-based: due unità, caso dipendente.

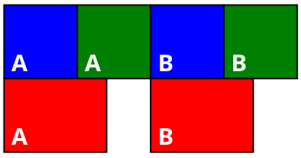

Figura 1.3: Task-based: due unità, caso indipendente.

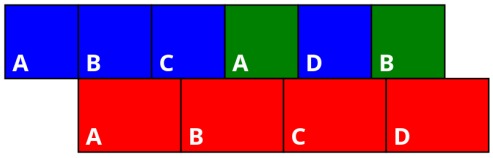

Figura 1.4: Task-based: due unità, più lavoro, caso dipendente.

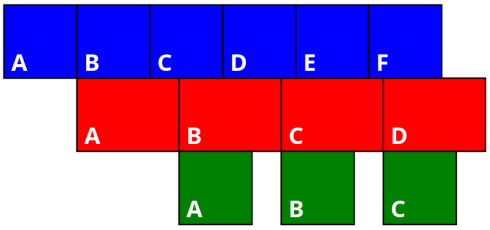

Figura 1.5: Task-based: tre unità, più lavoro, caso dipendente.

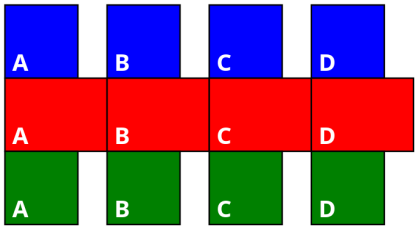

Figura 1.6: Task-based: tre unità, più lavoro, caso indipendente.

1.3.2 Data-based parallelism

Il data-based parallelism si basa sull'esecuzione di operazioni in parallelo su diversi dati. Tutte le unità computazionali fanno lo stesso lavoro, la divisione del lavoro avviene sui dati. Si preferisce questo approccio quando:

- Il lavoro può essere suddiviso in operazioni simili e semplici da eseguire su diversi dati.
- Si deve lavorare con una grossa mole di dati che può essere facilmente suddivisa in blocchi indipendenti.

Esempio: somma vettoriale. Un esempio classico di parallelismo *data-based* è la somma vettoriale. Consideriamo due vettori A e B di lunghezza N e un vettore risultato C, tale che:

$$C_i = A_i + B_i \qquad i = 0, \ldots, N-1$$

In questo caso l'operazione da eseguire è la stessa per tutti gli elementi, mentre cambiano solo i dati su cui essa viene applicata. Questo rende il problema particolarmente adatto al parallelismo data-based, poiché ogni elemento del vettore può essere elaborato indipendentemente dagli altri.

Seriale. - Nel caso **seriale** (Figura 1.7) un'unica unità di calcolo esegue tutte le operazioni in sequenza. Per ogni indice i vengono letti gli elementi A_i e B_i, viene calcolata la somma e il risultato viene scritto nella posizione corrispondente del vettore C. L'intero vettore viene quindi processato elemento per elemento.

Parallelo. - Nel caso **parallelo** (Figura 1.8), invece, più unità di calcolo eseguono la stessa operazione contemporaneamente su elementi diversi dei vettori. Ogni unità si occupa di un indice differente i, leggendo A_i e B_i, calcolando la somma e scrivendo il risultato in C_i. Poiché non esistono dipendenze tra le operazioni sui diversi elementi, tutte le somme possono essere eseguite in parallelo.

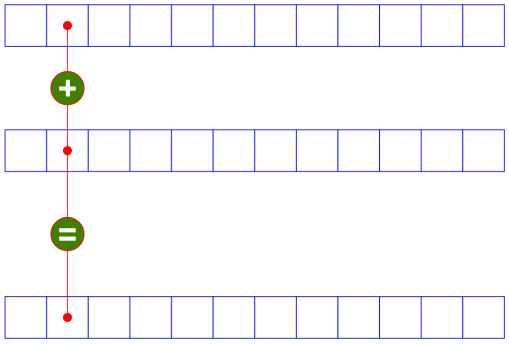

Figura 1.7: Somma vettoriale eseguita in modo seriale.

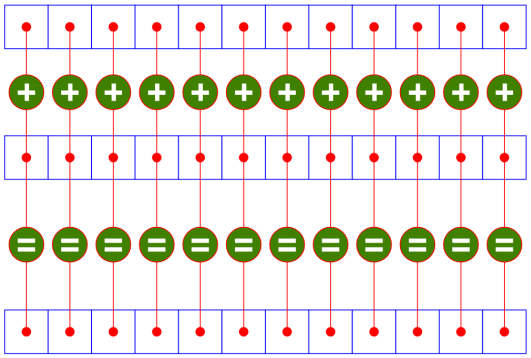

Figura 1.8: Somma vettoriale con parallelismo data-based.

1.4 Hardware per il calcolo parallelo

L'hardware per il calcolo parallelo è progettato per supportare l'esecuzione simultanea di più operazioni. Si può classificare l'hardware parallelo in base a come gestisce le sue risorse di calcolo e, in particolare, di memoria.

1.4.1 Shared memory

In un'architettura a memoria condivisa, tutte le unità di calcolo accedono alla stessa memoria centrale. Questo permette una comunicazione semplice tra le unità, poiché possono leggere e scrivere direttamente sui dati condivisi. Questo modello ha una comunicazione più semplice e meno latenza in generale, ma può essere limitato dalla contesa per l'accesso alla memoria e dalla scalabilità, poiché tutte le unità competono per le stesse risorse di memoria.

1.4.2 Distributed memory

In un'architettura a memoria distribuita, ogni unità di calcolo ha la propria memoria locale e non può accedere direttamente alla memoria delle altre unità. La comunicazione tra le unità avviene attraverso un sistema di messaggistica, in cui i dati devono essere esplicitamente inviati e ricevuti tra le unità. Questo modello è sicuramente più scalabile e può gestire grandi quantità di dati, ma la comunicazione tra le unità è più complessa e può introdurre latenza significativa, soprattutto se i dati devono essere scambiati frequentemente.

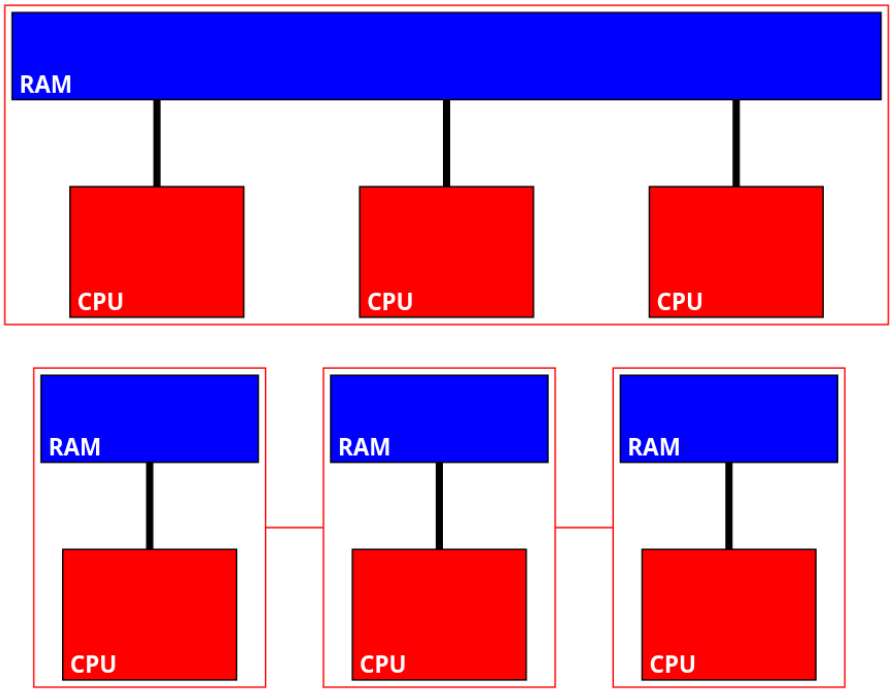

Figura 1.9: Confronto tra architettura a memoria condivisa (in alto), dove più CPU accedono alla stessa RAM, e architettura a memoria distribuita (in basso), dove ogni nodo possiede memoria locale e comunica con gli altri.

1.4.3 Ibrida

Un'architettura ibrida combina elementi di entrambe le architetture a memoria condivisa e distribuita. In questo modello, le unità di calcolo sono organizzate in gruppi, dove ogni gruppo condivide una memoria locale, ma i gruppi stessi comunicano tra loro attraverso un sistema di messaggistica. Questo approccio cerca di bilanciare i vantaggi di entrambi i modelli, offrendo una

maggiore scalabilità rispetto alla memoria condivisa, pur mantenendo una comunicazione più efficiente all'interno dei gruppi.

Un esempio sono i cluster di computer, dove ogni nodo del cluster ha la propria memoria locale, ma i nodi comunicano tra loro attraverso una rete ad alta velocità. All'interno di ogni nodo, le CPU possono accedere alla memoria condivisa, mentre tra i nodi la comunicazione avviene tramite messaggi.

1.5 Concorrenza, latenza e banda

Esistono misure di prestazioni per i sistemi paralleli che aiutano a comprendere l'efficienza e le limitazioni di un'architettura o di un algoritmo parallelo. Le principali sono:

Latenza: la latenza è il tempo necessario per completare un'operazione o una comunicazione. In un sistema parallelo, la latenza può essere influenzata da fattori come la contesa per le risorse, la comunicazione tra unità di calcolo e la sincronizzazione tra processi. Una bassa latenza è desiderabile per garantire che le operazioni vengano completate rapidamente e che le unità di calcolo non restino inattive in attesa di completare le operazioni.

Banda: la banda è la quantità di dati che può essere trasferita in un certo periodo di tempo. In un sistema parallelo, la banda è importante per garantire che i dati possano essere scambiati rapidamente tra le unità di calcolo, soprattutto in architetture a memoria distribuita. Una banda elevata è necessaria per evitare colli di bottiglia nella comunicazione e per garantire che le unità di calcolo possano accedere ai dati di cui hanno bisogno senza ritardi significativi.

Concorrenza: la concorrenza si riferisce alla capacità di un sistema di eseguire più operazioni contemporaneamente. In un sistema parallelo, la concorrenza è essenziale per sfruttare a pieno le risorse di calcolo disponibili. Tuttavia, la concorrenza può introdurre complessità nella gestione delle risorse, nella sincronizzazione tra processi e nella risoluzione di conflitti, soprattutto quando più processi accedono a dati condivisi. Per gestire la concorrenza in modo efficace esistono due soluzioni principali:

- Operazioni atomiche, ovvero operazioni che vengono eseguite in modo indivisibile, garantendo che nessun altro processo possa interferire durante la loro esecuzione.
- Semafori e/o mutex, che sono meccanismi di sincronizzazione che consentono di controllare l'accesso a risorse condivise, evitando conflitti e garantendo la coerenza dei dati.

Esempio: latenza vs banda. Ipotizziamo una connessione ad alta velocità, come *HyperTransport*, con il trasporto fisico di dati tramite un camion pieno di schede di memoria. Nel primo caso la bandwidth è di circa 50 GB/s con una latenza massima di circa 256 ns. Nel secondo caso, considerando un camion carico di schede microSD, la quantità totale di dati trasportabili può essere estremamente elevata, raggiungendo una bandwidth teorica superiore a 1000 TB/s. Tuttavia, il tempo necessario affinché i dati arrivino a destinazione è dell'ordine delle ore, risultando quindi in una latenza molto elevata.

Per migliorare le prestazioni dei sistemi paralleli è quindi importante adottare alcune strategie:

- evitare trasferimenti inutili, in modo da utilizzare la bandwidth in maniera più efficiente;

- accorpare più trasferimenti in operazioni più grandi, così da ammortizzare il costo della latenza.

1.6 Richiesta e offerta di calcolo parallelo

Il calcolo parallelo è diventato sempre più importante negli ultimi anni, sia per la crescente domanda di potenza di calcolo da parte di applicazioni complesse, sia per l'evoluzione dell'hardware che ha reso possibile l'esecuzione di operazioni in parallelo.

1.6.1 Domanda di calcolo parallelo

Generalmente in contesti **scientifici**, **industriali** e **commerciali** si richiede un'elevata potenza di calcolo per affrontare problemi complessi, analizzare grandi quantità di dati e sviluppare applicazioni avanzate. Gli usi principali sono:

- Monitoraggio: il calcolo parallelo consente di analizzare in tempo reale grandi quantità di dati provenienti da sensori, dispositivi IoT e sistemi di monitoraggio, permettendo di rilevare anomalie, prevedere guasti e ottimizzare le prestazioni.
- Simulazione: il calcolo parallelo è essenziale per eseguire simulazioni complesse in campi come la fisica, la chimica, l'ingegneria e la biologia, dove è necessario modellare fenomeni naturali, processi industriali o sistemi complessi.
- Analisi dei dati: il calcolo parallelo è fondamentale per l'analisi di grandi dataset, come quelli generati da social media, e-commerce, finanza e ricerca scientifica, consentendo di estrarre informazioni utili, identificare tendenze e prendere decisioni informate.

1.6.2 Offerta di calcolo parallelo

D'altra parte, l'offerta di calcolo parallelo è cresciuta significativamente grazie all'evoluzione dell'hardware e alla diffusione di tecnologie come i processori multi-core, le GPU e i cluster di computer. Queste tecnologie hanno reso possibile l'esecuzione di operazioni in parallelo su larga scala, offrendo una vasta potenza di calcolo. Oggi è possibile acquistare diverse soluzioni hardware per il calcolo parallelo, ciascuna con caratteristiche, vantaggi e limiti differenti.

Cluster: rappresentano la soluzione più tradizionale su cui si è sviluppato il *distributed computing*. Un cluster è composto da n nodi, ciascuno dotato di CPU multi-core e memoria locale, collegati tra loro tramite una rete ad alta velocità. Questa architettura è ampiamente utilizzata in ambito scientifico e industriale per applicazioni che richiedono elevate capacità di calcolo.

I cluster costituiscono una soluzione classica e consolidata nel tempo, supportata da un vasto ecosistema software e da una grande diffusione di competenze tecniche. Offrono inoltre un'elevata flessibilità, poiché è possibile adattare l'infrastruttura a diverse tipologie di applicazioni.

D'altra parte, presentano anche alcuni svantaggi significativi. L'investimento iniziale in termini di hardware può essere molto elevato e i costi di manutenzione risultano spesso importanti, soprattutto per quanto riguarda il consumo energetico e i sistemi di raffreddamento.

Inoltre il rapporto costo/prestazioni può risultare relativamente alto e l'efficienza energetica, misurata come prestazioni per watt consumato, non è sempre ottimale.

GPU: le unità di elaborazione grafica sono nate originariamente per l'elaborazione grafica nei videogiochi, ma nel tempo si è scoperto che la loro architettura altamente parallela poteva essere sfruttata anche per il calcolo generale. Da questa intuizione è nato il paradigma *GPGPU* (*General-Purpose computing on GPU*), che consente di utilizzare le GPU per eseguire calcoli scientifici e applicazioni ad alte prestazioni.

Le GPU offrono diversi vantaggi: il costo iniziale dell'hardware è generalmente inferiore rispetto a soluzioni basate su grandi cluster, così come i costi di manutenzione. Inoltre presentano un rapporto prezzo/prestazioni molto favorevole e un'elevata efficienza energetica, rendendole particolarmente adatte per applicazioni come machine learning, simulazioni numeriche e analisi di grandi quantità di dati.

Tuttavia, si tratta di una tecnologia relativamente più recente rispetto ai cluster tradizionali. Il modello di programmazione GPGPU ha iniziato a diffondersi solo a partire dalla fine del 2006 e dall'inizio del 2007, e per questo motivo l'ecosistema software è storicamente meno maturo. In alcuni contesti sono ancora disponibili meno strumenti, meno applicazioni consolidate e una minore diffusione di competenze specifiche, oltre a una flessibilità talvolta inferiore rispetto alle architetture più tradizionali.

Acceleratori: dispositivi specializzati che affiancano la CPU per eseguire operazioni specifiche in modo altamente parallelo. Questi componenti vengono utilizzati per migliorare le prestazioni in determinati ambiti applicativi, come l'elaborazione di segnali, la crittografia, il machine learning o il calcolo scientifico ad alte prestazioni. Gli acceleratori permettono di delegare alcune operazioni particolarmente intensive a hardware dedicato, aumentando l'efficienza complessiva del sistema.

1.6.3 Videogiochi e GPU

I videogiochi rappresentano un esempio di applicazione che richiede un'elevata potenza di calcolo parallelo, soprattutto per quanto riguarda l'elaborazione grafica. Le GPU sono state sviluppate proprio per soddisfare questa esigenza, consentendo di renderizzare scene complesse in tempo reale. I giochi moderni devono supportare:

- Gameplay più sofisticato
- Grafiche più complesse
- Tre dimensioni
- Fisica più realistica

e per farlo devono sfruttare software e hardware migliori.

Esempio: software migliore. Un esempio storico di ottimizzazione software molto noto nel contesto dei videogiochi è l'algoritmo per il calcolo rapido dell'inversa della radice quadrata, utilizzato per velocizzare operazioni molto frequenti nella grafica 3D, come la normalizzazione dei vettori.

```
float rsqrt(float x)
{
    float h = x/2;
    int i = *(int*)&x;
    i = 0x5f3758d - (i>>2);
    x = *(float*)&i;
    return x*(1.5f - h*x*x); /* newton */
}
```

Listing 1.1: Implementazione approssimata della radice quadrata inversa

Questo codice implementa una stima rapida di $1/\sqrt{x}$. L'idea è ottenere una prima approssimazione tramite una manipolazione diretta della rappresentazione binaria del numero floating-point e poi migliorarla con un'iterazione del metodo di Newton. L'efficienza di questo approccio deriva dal fatto che evita il calcolo diretto della radice quadrata, che storicamente era molto più costoso dal punto di vista computazionale.

Esempio: hardware migliore. Per ottenere hardware migliore si possono seguire diverse strategie:

- CPU più veloci, con frequenze di clock più elevate e architetture più efficienti. Questo ha un costo elevato e limita la scalabilità, poiché esistono limiti fisici alla velocità di clock e all'efficienza energetica delle CPU.
- Più dischi rigidi, per aumentare la capacità di archiviazione e migliorare le prestazioni di I/O. Tuttavia, questo approccio non migliora direttamente la potenza di calcolo e può introdurre colli di bottiglia nella gestione dei dati.
- Più memoria RAM e cache, per ridurre i tempi di accesso ai dati e migliorare le prestazioni complessive del sistema. Anche questo approccio ha limiti in termini di costo e scalabilità, poiché la memoria è una risorsa costosa e l'efficienza energetica può diminuire con l'aumentare della quantità di memoria.
- Hardware video dedicato, come le GPU, che offrono un'architettura altamente parallela e sono progettate per gestire carichi di lavoro specifici come l'elaborazione grafica. Questo approccio è particolarmente efficace per migliorare le prestazioni nei videogiochi, ma può essere meno flessibile rispetto a soluzioni più generiche come i cluster di CPU.

1.7 Storia delle GPU

1.7.1 Dai primi adattatori video alle prime schede programmabili

L'evoluzione delle GPU può essere interpretata come un progressivo aumento di quattro aspetti fondamentali: **risoluzione**, **numero di colori**, **accelerazione hardware** e **programmabilità**. Le prime schede video erano semplici adattatori grafici: **CGA** (1981) supportava 640×200 in monocromatico oppure 320×200 con 4 colori, oltre a modalità testuali con 16 colori. Con **EGA** (1984) si passa a 640×350 con una palette di 16 colori scelta da un insieme di 64, ottenendo quindi una migliore qualità visiva sia in termini di dettaglio sia di rappresentazione cromatica.

Un salto più importante si ha con **VGA** (1987), che introduce 6 bit per colore per pixel, fino a 262 144 colori rappresentabili, e modalità standard come 640×480 con 16 colori da una palette di

256 e 320×200 con 256 colori. VGA viene anche considerato il primo adattatore video pienamente programmabile. Nello stesso periodo compaiono soluzioni come **IBM 8514**, che oltre a supportare 1024×768 con 256 colori in modalità interlacciata introducono l'accelerazione hardware di primitive 2D, come il disegno di linee, il riempimento di aree e il *bit blit*. Con **XGA** (1990) si consolidano ulteriormente questi miglioramenti, con modalità come 800×600 a 16 bit di colore e 1024×768 con palette a 256 colori.

1.7.2 La transizione verso la grafica 3D

Negli anni Novanta, la diffusione delle interfacce grafiche e soprattutto dei videogiochi 3D cambia il ruolo delle schede video: non devono più soltanto visualizzare immagini, ma accelerare trasformazioni geometriche, illuminazione e rendering di scene tridimensionali. Nascono così le prime schede dedicate al 3D, che ottengono grande successo commerciale grazie a una grafica più pulita e animazioni più fluide.

Per rendere l'accesso a questo hardware più uniforme si affermano API standard come **OpenGL** e **DirectX**. Con l'introduzione del **programmable shading**, le schede video smettono di essere pipeline fisse e diventano dispositivi progressivamente programmabili: è in questo contesto che si diffonde il termine **GPU**.

1.7.3 Dalla GPU al GPGPU

Da qui nasce anche l'idea di utilizzare la GPU non solo per la grafica ma anche per il calcolo generale. In una prima fase, intorno al 2004, i problemi numerici venivano trasformati artificialmente in problemi grafici e risolti tramite il rendering, in modo complesso ma spesso efficace. La vera svolta arriva quando i produttori rendono accessibile in modo diretto la potenza di calcolo delle GPU: **ATI/AMD** con **CAL** e **NVIDIA** con **CUDA**. Da questo momento la GPU non è più soltanto un acceleratore grafico, ma una vera piattaforma di calcolo parallelo programmabile.

1.8 GPU moderne

Oggi le GPU sono dispositivi estremamente potenti e versatili, utilizzati non solo per la grafica ma anche per una vasta gamma di applicazioni scientifiche, industriali e commerciali. Le GPU moderne sono caratterizzate da un'architettura altamente parallela, si parla di centinaia di multiprocessori ognuno equipaggiato con circa 100 **Processing Unit** (come i CUDA core di NVIDIA), che consentono di eseguire migliaia di thread contemporaneamente. Una processing unit è un'unità di calcolo che esegue operazioni aritmetiche e logiche sui dati, ed è progettata per gestire carichi di lavoro altamente paralleli.

1.8.1 GPU e GPU: performance e costi

Le GPU e le CPU sono progettate per scopi diversi e presentano caratteristiche architetturali differenti, che is riflettono molto sulle prestazioni finali. Le GPU sono ottimizzate per eseguire un gran numero di operazioni in parallelo, con un'architettura che consente di gestire migliaia di thread contemporaneamente. Le CPU, invece, sono progettate per eseguire operazioni più complesse e meno parallelizzabili, con un numero limitato di core e una maggiore enfasi sulla latenza.

Dalle tabelle riportate in Tabella 1.2 si può notare che le GPU offrono un numero significativamente maggiore di unità di elaborazione (MPs) e una maggiore potenza di calcolo in termini di FLOPS rispetto alle CPU. Inoltre, le GPU tendono ad avere una banda di memoria molto più elevata rispetto alle CPU, il che è cruciale per gestire i grandi volumi di dati necessari per le applicazioni di calcolo parallelo. Tuttavia, è importante considerare anche i costi associati all'acquisto e alla manutenzione di questi dispositivi, nonché la complessità dell'ecosistema software e la disponibilità di competenze specifiche per sfruttare a pieno le potenzialità delle GPU.

GPU vs CPU: performance

Model	MPs	PE/MP	FLOPS (TFLOPS)	bandwidth (GB/s)
AMD Radeon VII	60	64	11.1	1028
NVIDIA TITAN RTX	72	64	12.4	672

Model	cores	PE/core	FLOPS (GFLOPS)	bandwidth (GB/s)
AMD EPYC 7702P 2GHz	64	16	4096	143.1
Intel Xeon W-3275M 2.5GHz	28	32	3072	131.13

Tabella 1.2: Confronto prestazionale tra alcune GPU e CPU.

APU. Una terza opzione è rappresentata dalle APU (**Accelerated Processing Unit**), che integrano CPU e GPU su un unico chip. Le APU offrono un compromesso tra le prestazioni di calcolo parallelo delle GPU e la flessibilità delle CPU, consentendo di eseguire sia operazioni generali che specifiche per la grafica in modo efficiente. Tuttavia, le APU tendono ad avere prestazioni inferiori rispetto a soluzioni dedicate come le GPU discrete, ma possono essere una scelta interessante per applicazioni che richiedono un equilibrio tra potenza di calcolo e costi.

1.9 Parallelizzazione

1.9.1 Concetto generale di parallelizzazione

La parallelizzazione è il processo di suddivisione di un problema o di un algoritmo seriale in un algoritmo parallelo. Ipotizziamo due estremi:

Un algoritmo intrinsecamente seriale è un algoritmo che non può essere parallelizzato, poiché ogni operazione dipende, per esempio, strettamente dai risultati delle operazioni precedenti. In questo caso, l'unico modo per eseguire l'algoritmo è in modalità sequenziale, e lo speed-up ottenibile con il calcolo parallelo è limitato. Tuttavia un modo per parallelizzare un algoritmo seriale è quello di fare parallelismo *data-based*, ovvero eseguire in parallelo operazioni simili su dati diversi, anche se l'algoritmo stesso non è parallelizzabile.

Un algoritmo intrinsecamente parallelo è un algoritmo che può essere suddiviso in parti indipendenti che possono essere eseguite contemporaneamente. In questo caso, è possibile ottenere uno speed-up significativo utilizzando più unità di calcolo, poiché le operazioni possono essere eseguite in parallelo senza dipendenze.

1.9.2 Algoritmi con parallelismo parziale

Esistono però vie di mezzo, ovvero algoritmi che non sono intrinsecamente seriali ma che presentano una certa quantità di parallelismo. Alcuni esempio sono:

- Riduzione: operazioni che combinano più elementi in un unico risultato, come la somma di un array o il prodotto di matrici. Questi algoritmi possono essere parallelizzati suddividendo i dati in blocchi e combinando i risultati parziali.
- Scan: operazioni che producono un array di risultati parziali, come la somma cumulativa o il prefisso di un array. Questi algoritmi possono essere parallelizzati utilizzando tecniche come il *divide and conquer* per suddividere i dati e combinare i risultati.
- Algoritmi di ordinamento: algoritmi come il merge sort o il quicksort possono essere parallelizzati suddividendo l'array in parti più piccole e ordinando ciascuna parte in parallelo, per poi combinare i risultati.

1.9.3 Parallelismo in hardware

Dal punto di vista dell'hardware, esistono diversi modelli con cui il parallelismo può essere implementato. I principali sono:

SIMD (*Single Instruction, Multiple Data*) indica un modello in cui una singola istruzione opera contemporaneamente su più dati. Questo approccio è tipico delle istruzioni vettoriali presenti nelle CPU moderne, come MMX, AltiVec, SSE, NEON o AVX, ed è particolarmente efficace quando si devono eseguire gli stessi calcoli su vettori o array.

SPMD (*Single Program, Multiple Data*) indica un modello nel quale la stessa sequenza di istruzioni viene eseguita da più unità di calcolo contemporaneamente su dati differenti. In questo caso non si parla necessariamente di una singola istruzione eseguita nello stesso istante, ma dello stesso programma applicato a porzioni diverse dell'input. Questo modello può essere supportato sia dall'hardware sia dal software.

SIMT (*Single Instruction, Multiple Threads*) è un modello tipico delle GPU. In questo caso l'hardware gestisce automaticamente gruppi di thread che eseguono la stessa istruzione in parallelo. Si tratta quindi di un modello vicino a SPMD, ma con una gestione dei thread fortemente integrata a livello hardware.

In generale, si può osservare che le **GPU** moderne sono tipicamente associate ai modelli **SPMD** e **SIMT**, mentre le **CPU** moderne sono principalmente **SIMD**, eventualmente affiancate da forme di SPMD controllate dal software.

1.9.4 Parallelismo in software

Dal punto di vista software, il parallelismo può essere organizzato secondo modelli differenti. I principali sono:

Multi-processing è il parallelismo basato su più thread in memoria condivisa. In questo caso un singolo programma genera più thread che operano sullo stesso spazio di memoria condiviso. Questo modello è tipico dei sistemi multicore e lo standard più diffuso su CPU è **OpenMP**.

Message-passing è un modello nel quale più processi distinti comunicano tra loro scambiandosi messaggi. Questo approccio è naturale nei sistemi a memoria distribuita, dove ogni processo possiede una propria memoria locale e la cooperazione avviene tramite comunicazione esplicita. Lo standard di riferimento in questo ambito è **MPI** (*Message Passing Interface*).

Hybrid combina i due modelli precedenti: più processi comunicano tramite messaggi e, all'interno di ciascun processo, sono presenti più thread. Questo modello è molto comune nei sistemi HPC moderni, poiché consente di sfruttare contemporaneamente il parallelismo interno ai nodi e quello tra nodi diversi.

Capitolo 2

Fondamenti di GPGPU

2.1 Architettura di un programma GPGPU

L'architettura di un programma GPGPU è composta da tre componenti principali:

Host Il processore centrale (CPU) che esegue il codice principale del programma e gestisce la comunicazione con la GPU.

Device La GPU, che esegue i kernel (flussi di esecuzione parallela) e gestisce la memoria dedicata.

Compute Unit (CU) Un'unità di elaborazione all'interno della GPU che esegue i kernel in parallelo. Ogni CU contiene più core di elaborazione, che possono eseguire kernel in parallelo.

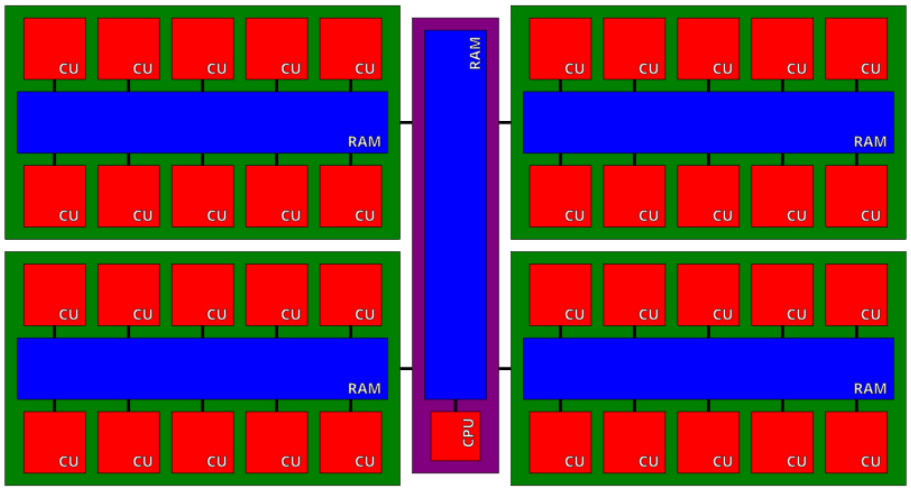

Figura 2.1: Architettura di un sistema OpenCL, con CPU in viola(host) e diverse GPU (device) in verde che comunicano tra loro.

2.1.1 Kernel, work-item e work-group

Un kernel è un flusso di lavoro che viene eseguita in parallelo su più compute unit della GPU. Ogni istanza di esecuzione del kernel è detta **work-item** (in CUDA vengono chiamati "thread") e ogni work-item che appartengono allo stesso **work-group** (un gruppo di work-item) condividono la memoria locale e possono sincronizzarsi tra loro.

La collezione di tutti i work-item che eseguono un kernel è chiamata **launch grid** (o, in OpenCL, "NDRange"). La dimensione del launch grid è definita in termini di numero di work-item e work-group, e può essere configurata per massimizzare l'efficienza dell'esecuzione del kernel sulla GPU.

2.2 Stream-Processing

La prima forma di parallelismo su GPU è lo *stream-processing*, in cui i dati vengono elaborati in flussi (streams) e ogni elemento del flusso viene elaborato in parallelo. Questo modello è adatto per operazioni che possono essere eseguite indipendentemente su ciascun elemento, come ad esempio l'elaborazione di immagini o la simulazione di particelle.

2.2.1 Kernel e dati

In questo caso il kernel è la sequenza di istruzioni da eseguire su porzioni corrispondenti di più input stream per ottenere uno stream di output. Il kernel lavora solo con i dati che gli sono stati assegnati, e non ha accesso a dati al di fuori del suo work-item o work-group.

Inoltre il programmatore non ha alcun controllo sulla divisione del lavoro tra i work-item o tra i work-group e non può sincronizzare i work-item tra loro, se non quelli appartenenti allo stesso work-group.

2.2.2 Esempio: Mandelbrot fractal

Il frattale di Mandelbrot è un insieme di punti complessi che non diverge quando iterati attraverso una funzione quadratica. Per generare un'immagine del frattale, ogni pixel dell'immagine può essere calcolato indipendentemente, rendendo questo problema ideale per il parallelismo su GPU.

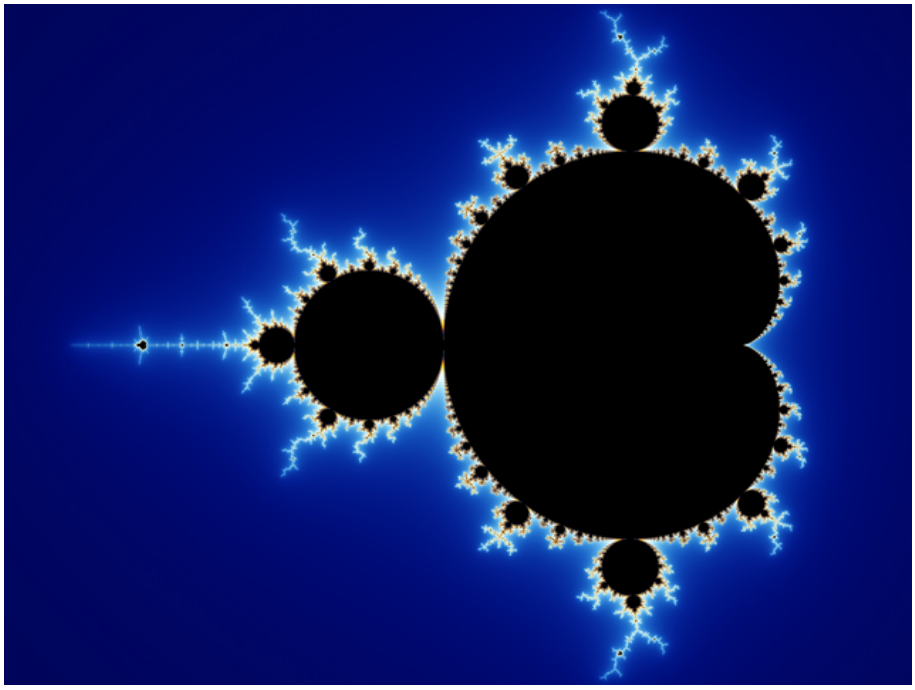

Figura 2.2: Frattale di Mandelbrot generato utilizzando il parallelismo su GPU.

Si può rapresentare la serie di questo frattale come

$$z_{n+1} = z_n^2 + c$$

dove z è un numero complesso e c è un punto complesso che rappresenta la posizione del pixel nell'immagine. Il kernel calcola il numero di iterazioni necessarie affinché z diverga (superi un certo limite) per ogni pixel, e questo numero di iterazioni viene utilizzato per determinare il colore del pixel nell'immagine finale.

Versione seriale. La versione seriale del codice per generare il frattale di Mandelbrot potrebbe essere scritta come:

```
int nrows, ncols, maxiter;
float xmin, xmax, ymin, ymax;
int *bitmap;

for (int row = 0; row < nrows; ++row) {
    for (int col = 0; col < ncols; ++col) {
        complex z, c;
        int iter;

        c.real = xmin + col*(xmax - xmin)/ncols;
        c.img  = ymin + row*(ymax - ymin)/nrows;

        z = c;

        for (iter = 0; iter < maxiter; ++iter) {
            z = add(mul(z, z), c); /* z_{n+1} = z_n^2 + c */
            if (norm(z) > 2)
                break;
        }

        bitmap[row*ncols + col] = !!(iter < maxiter);
    }
}
```

Listing 2.1: Programma seriale per il calcolo dell'insieme di Mandelbrot

Il core parallelizzabile di questo codice è il doppio ciclo annidato che itera su ogni pixel dell'immagine. Ogni iterazione del ciclo interno calcola il numero di iterazioni necessarie affinché z diverga per un pixel specifico, e questo calcolo può essere eseguito in parallelo per tutti i pixel dell'immagine. Questo può essere considerato un candidato ideale per scrivere un kernel che esegue il calcolo in parallelo su una GPU, dove ogni work-item si occupa di calcolare il numero di iterazioni per un pixel specifico.

2.3 Memoria

I device di calcolo hanno la propria memoria dedicata, che è separata dalla memoria del host. Gli spazi di memoria presenti sulla GPU sono diversi:

Global Memory La memoria globale è accessibile a tutti i work-item e work-group, ma ha una latenza elevata. I contenuti in essi presenti permangono per la durata di esecuzione dell'intero programma e i dati possono essere modificati dai kernel.

Constant Memory La memoria costante è una porzione di memoria globale che è ottimizzata per il broadcasting. Può essere modificata solo dall'host e se necessario può essere cambiata tra l'esecuzione di un kernel e l'altro.

Images (o texture in CUDA) La memoria per le immagini è ottimizzata per l'accesso 2D e 3D, e supporta operazioni di filtraggio, interpolazione, l'indexing multidimensionale e a normalizzazione delle coordinate.

Local Memory (shared memory in CUDA) La memoria locale si trova fisicamente sulle compute unit e, a differenza della memoria globale, è volatile e viene allocata all'inizio di un determinato kernell e successivamente svuotata. Può essere comunque utilizzata dai work-item appartenenti allo stesso work-group per condividere dati e sincronizzarsi tra loro. Viene utilizzata per algoritmi più complessi che richiedono la condivisione di dati tra i work-item, come ad esempio le operazioni di riduzione o di convoluzione.

Private memory (local memory in CUDA) Questa memoria contiene dati temporanei specifici di un singolo work-item, tipitcamente i dati da inserire all'interno dei registri di esecuzione. Se non vi sono registri sufficienti per la quantità di dati da gestire si ha un *register spill*[1] e i dati vengono spostati in local memory, con conseguente degrado delle prestazioni.

2.4 Struttura di un programma GPGPU

2.4.1 Pipeline di esecuzione

Un programma GPGPU è composto da 4 parti principali:

Selects. - Selezione del dispositivo di calcolo (GPU) su cui eseguire il programma. In questo caso CUDA fa tutto a runtime dietro le quinte, mentre OpenCL richiede al programmatore di specificare esplicitamente il dispositivo da utilizzare. Esempio: se si dispone di più GPU, è possibile selezionare quella più adatta per l'esecuzione del programma, ad esempio in base alla quantità di memoria disponibile o alla potenza di calcolo.

Uploads. - Trasferimento dei dati dal host alla GPU. Questo passaggio è necessario perché la GPU ha la propria memoria dedicata e non può accedere direttamente alla memoria del host. Esempio: se si sta elaborando un'immagine, è necessario caricare i dati dell'immagine nella memoria della GPU prima di eseguire il kernel che elabora l'immagine.

Runs. - Esecuzione del kernel sulla GPU. In questo passaggio, il kernel viene lanciato sulla GPU e i work-item eseguono il codice in parallelo. Esempio: se si sta calcolando il frattale di Mandelbrot, è necessario lanciare un kernel che esegue il calcolo per ogni pixel dell'immagine in parallelo.

[1] Il register spill è un fenomeno che si verifica quando un kernel utilizza più registri di quelli disponibili sulla GPU, costringendo il compilatore a spostare alcuni dati dalla memoria dei registri alla memoria globale o locale. Questo può causare un degrado delle prestazioni, poiché l'accesso alla memoria globale o locale è molto più lento rispetto all'accesso ai registri.

Downloads. - Trasferimento dei dati dalla GPU al host. Dopo l'esecuzione del kernel, i risultati devono essere trasferiti nuovamente al host per essere utilizzati o visualizzati. Esempio: se si sta elaborando un'immagine, è necessario scaricare i dati dell'immagine elaborata dalla memoria della GPU al host per visualizzarla o salvarla su disco.

2.4.2 Comunicazione host-device

Considerando un setup moderno:

- Bandwidth host: 50 GB/s (PCIe 4.0 x16)
- Bandwidth device: 900 GB/s (HBM2)

Il problema principale di questo tipo di architettura è che la comunicazione tra host e device è molto più lenta rispetto alla comunicazione all'interno del device stesso. Pertanto, è importante minimizzare il numero di trasferimenti di dati tra host e device per massimizzare le prestazioni del programma GPGPU.

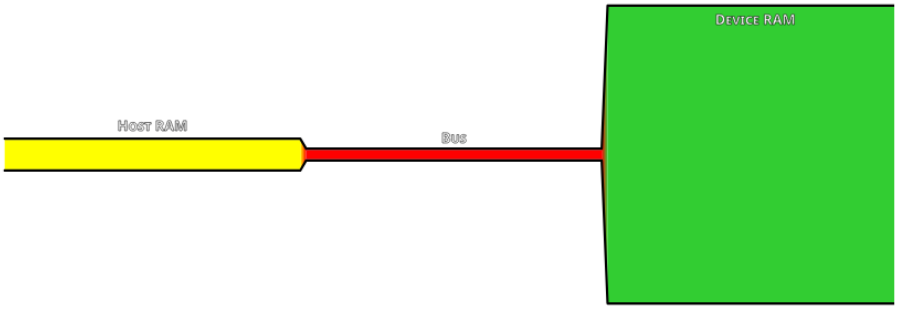

Figura 2.3: Comunicazione tra host e device, con i trasferimenti di dati che avvengono attraverso il bus PCIe.

2.4.3 Sorgente singolo e separato

Come già accennato, un programma GPGPU è composto da due parti distinte: il codice *host*, eseguito sulla CPU, e il codice *device*, costituito dai kernel che verranno eseguiti sulla GPU. Queste due componenti non solo vengono eseguite su architetture differenti, ma sono anche tradotte in linguaggi macchina distinti; inoltre, tale linguaggio può variare anche tra diverse generazioni di GPU dello stesso produttore.

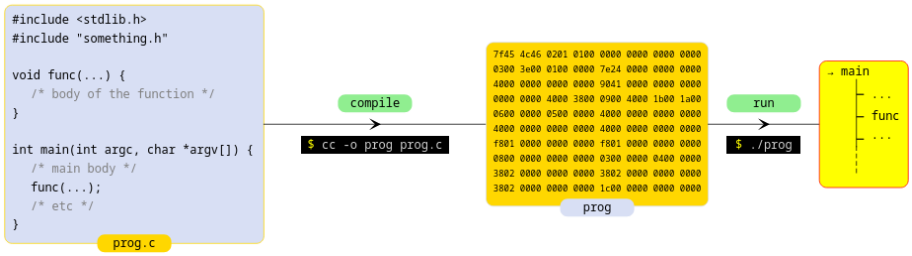

Figura 2.4: Processo di compilazione ed esecuzione di un programma C: dal codice sorgente alla generazione dell'eseguibile e alla sua esecuzione.

Il modo in cui queste due parti vengono organizzate e compilate porta a distinguere due approcci principali: il modello *single-source* e il modello *separate-source*.

Single-source. Nel modello *single-source*, tipico ad esempio di CUDA, il codice host e quello device convivono all'interno dello stesso file sorgente. In questo caso è il compilatore a occuparsi automaticamente di separare le due componenti e di gestirne la compilazione.

Più precisamente, le funzioni destinate all'esecuzione sulla GPU vengono identificate tramite annotazioni specifiche, come `__global__`. Il compilatore (ad esempio `nvcc`) analizza il sorgente e lo suddivide in una parte host, che viene passata al compilatore tradizionale, e una parte device, che viene compilata con un backend dedicato alla GPU. Il risultato è la produzione di due codici macchina distinti, uno per la CPU e uno per il device.

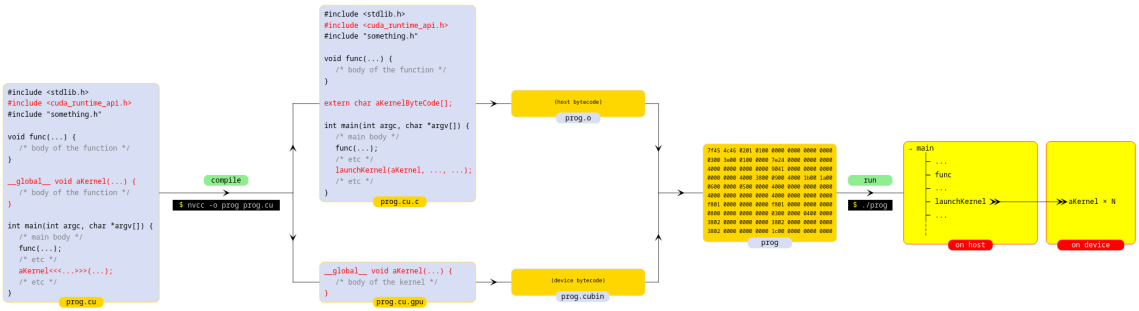

Figura 2.5: Processo di compilazione ed esecuzione in CUDA: il codice single-source viene separato in componente host e device, compilato in bytecode distinti e ricombinato in un unico eseguibile che coordina CPU e GPU.

Dal punto di vista dell'esecuzione, il codice host può invocare un kernel tramite una sintassi dedicata, come `aKernel«<...»>();`, specificando la configurazione della griglia di esecuzione. È importante osservare che il lancio del kernel è asincrono rispetto all'host: questo significa che l'host continua la propria esecuzione senza attendere il completamento del kernel, e quindi una misura del tempo attorno al lancio non rappresenta il tempo reale di esecuzione sul device.

Questo approccio risulta generalmente più semplice da utilizzare, in quanto riduce la quantità di codice da gestire e garantisce coerenza tra i tipi di dato utilizzati lato host e lato device. Tuttavia, introduce alcuni vincoli: richiede l'uso di un compilatore specifico, che deve essere compatibile con il compilatore host, e impone di specificare a priori il device di destinazione. Inoltre, non consente una compilazione selettiva dei singoli kernel.

Separate-source. Nel modello *separate-source*, adottato ad esempio in OpenCL, il codice device è completamente separato da quello host e viene gestito come entità indipendente.

In questo caso, i kernel risiedono tipicamente in file distinti oppure vengono rappresentati come stringhe all'interno del codice host. Durante l'esecuzione, è il programma host a caricare il sorgente dei kernel e a richiederne la compilazione al driver della GPU. La compilazione avviene quindi a runtime, e non in fase di build, rendendo questo approccio più dinamico e flessibile. I

driver, per ottimizzare le prestazioni, mantengono generalmente una cache dei kernel già compilati, evitando di ripetere operazioni costose.

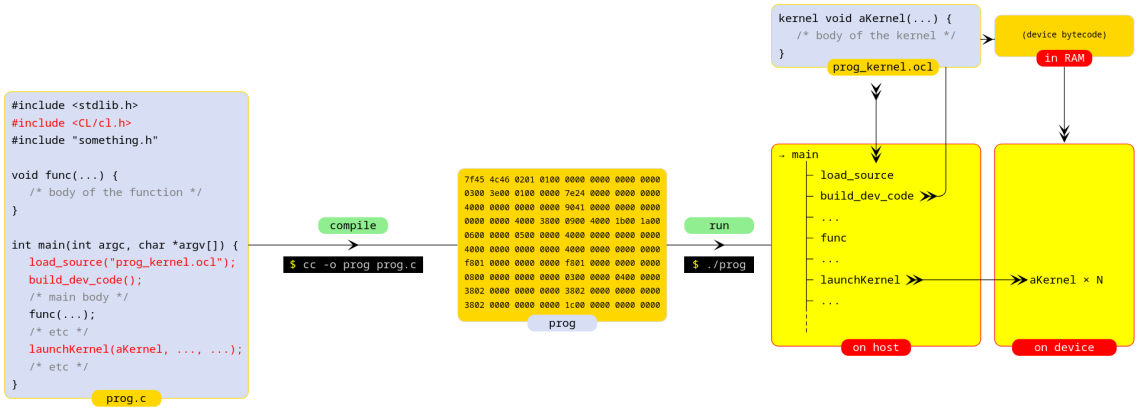

Figura 2.6: Processo di compilazione ed esecuzione in OpenCL: il codice host viene compilato separatamente, mentre i kernel device sono caricati e compilati a runtime dal driver prima dell'esecuzione sulla GPU.

Questo modello offre una maggiore portabilità, in quanto lo stesso codice device può essere compilato su architetture differenti e non richiede l'uso di un compilatore specifico per l'host. Inoltre, permette di selezionare dinamicamente quali kernel compilare ed eseguire.

D'altra parte, questa flessibilità comporta un aumento della complessità: il codice da gestire è maggiore e la separazione tra host e device rende meno immediata la condivisione dei tipi di dato, che devono quindi essere definiti con particolare attenzione, privilegiando strutture semplici e compatibili tra i due ambienti.

2.5 Stream Computing